LECTURE – 6

Definition of the sorting problem:

Input: Sequence of n numbers a_1, a_2 .... a_n

Output: A *permutation* (*reordering*) of the input sequence s. t. numbers in the output sequence are monotonically increasing (or decreasing).

NOTE: The input and output sequences may be implemented in the form of arrays, linked lists, or other suitable structures.

Points to note:

1. Meaning of record, key, satellite data. Implications for implementation.

2. Sorting algorithms are very often embedded in larger applications.

3. Many useful techniques need to be mastered while studying and implementing sorting algorithms, including running time analysis.

4. In practice, the hardware and software characteristics must also be considered so as to achieve optimum performance.

5. Worst case, best case, average case running time; theoretical lower bound on worst case running time of sorting algorithms based on comparisons.

6. Order statistics.

HEAPSORT

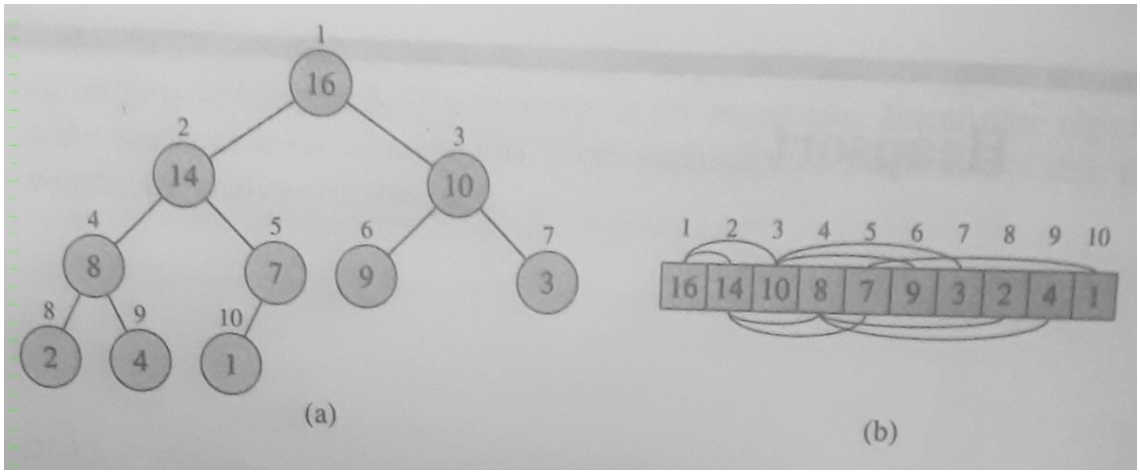
This sort algorithm is centred around the data structure known as **HEAP**.

Definition: [based on CLRS]

An array object that can be viewed as a nearly complete binary tree. Every node of the tree corresponds to an element in array A which has a number a_k stored in it, $1 \leq k \leq n$. The data structure satisfies the 'heap property'.

Notation: length[A] is the array size, heap-size[A] $\leq$ length[A] is the number of nodes in the heap.

The first heap-size[A] elements of the array contain tree nodes, the rest are unused.



Heap property: The following relationships are satisfied at all times:

parent(i) is given by floor(i/2) //parent node, except for root node

left(i) is given by 2i //left child node, if any

right(i) is given by 2i+1 //right child node, if any

Two types of heap: max-heap and min-heap

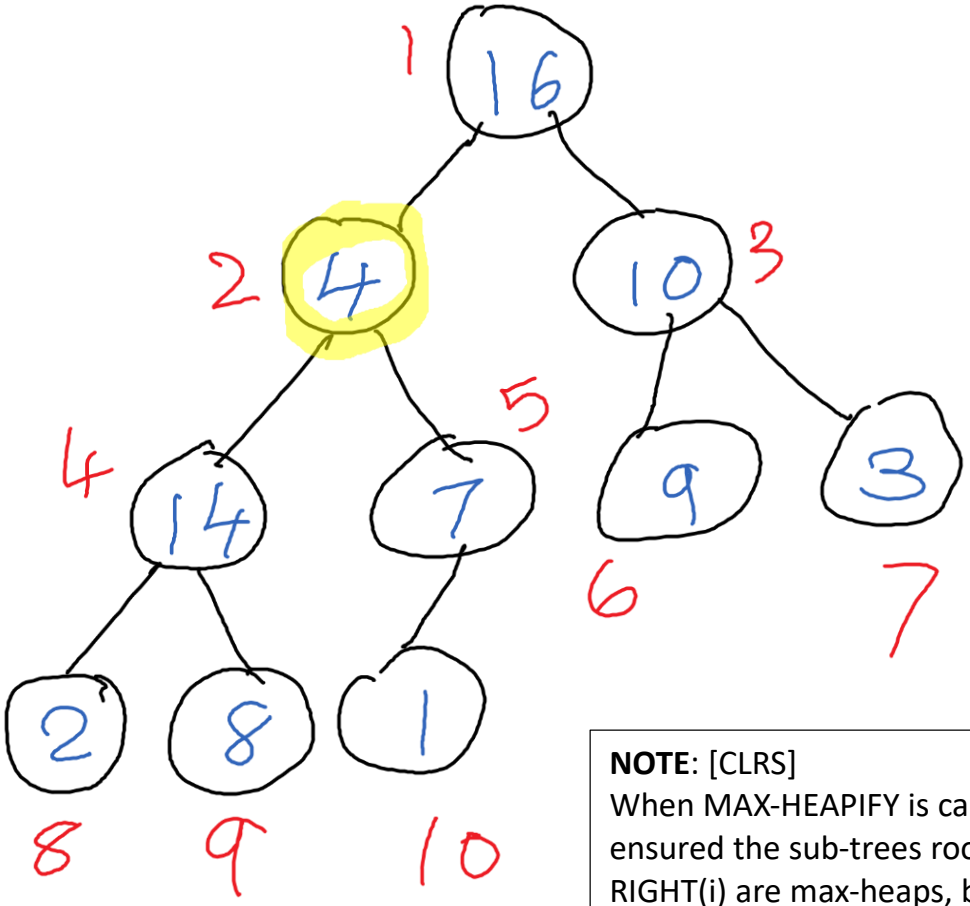
Max-heap property: A[parent(i)] $\geq$ A[i]

Min-heap property: A[parent(i)] $\leq$ A[i]

The heap data structure is useful in many ways, including in sorting.

Now we consider the key function which maintains the heap property, with a simple example of the application of that function.

```
MAX-HEAPIFY(A,i)
  l = LEFT(i);
  r = RIGHT(i);
  if l ≤ heap-size[A] && A[l] > A[i]
  then
    largest = l;
  else
    largest = i;
  if r ≤ heap-size[A] && A[r] > A[largest]
    largest = r;
  if largest <> i
  then
    exchange A[i] and A[largest];
    MAX-HEAPIFY(A,largest);     //Note recursive call
```



NOTE: [CLRS]
When MAX-HEAPIFY is called, it should be ensured the sub-trees rooted at LEFT(i) and RIGHT(i) are max-heaps, but A[i] may be smaller than one or both of its child nodes. In other words, the only possible violation of the max-heap property is at node i.